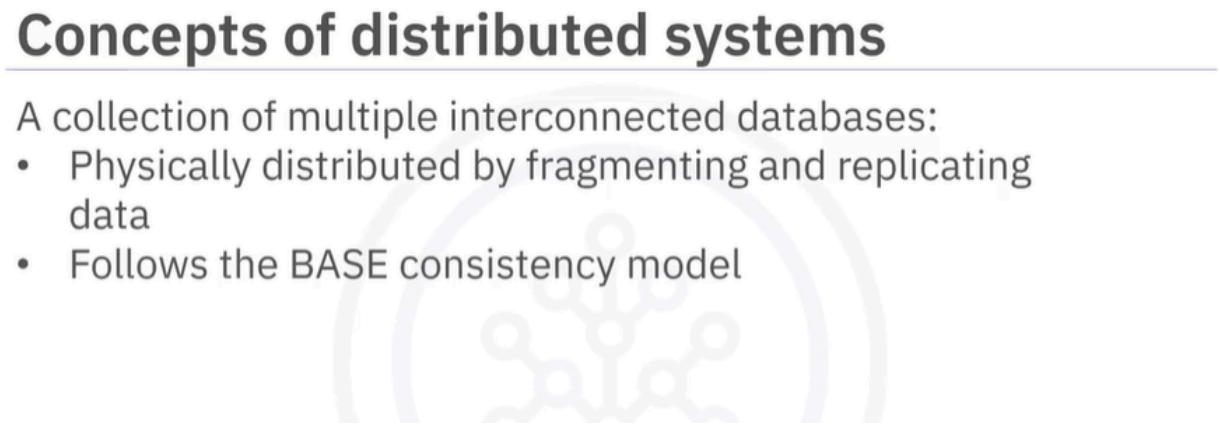
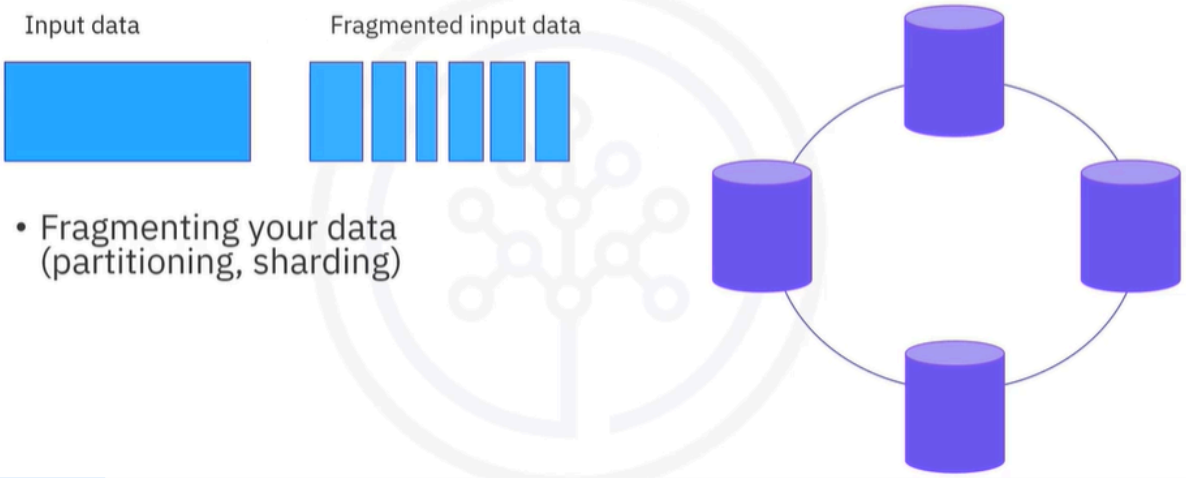
Distributed Databases



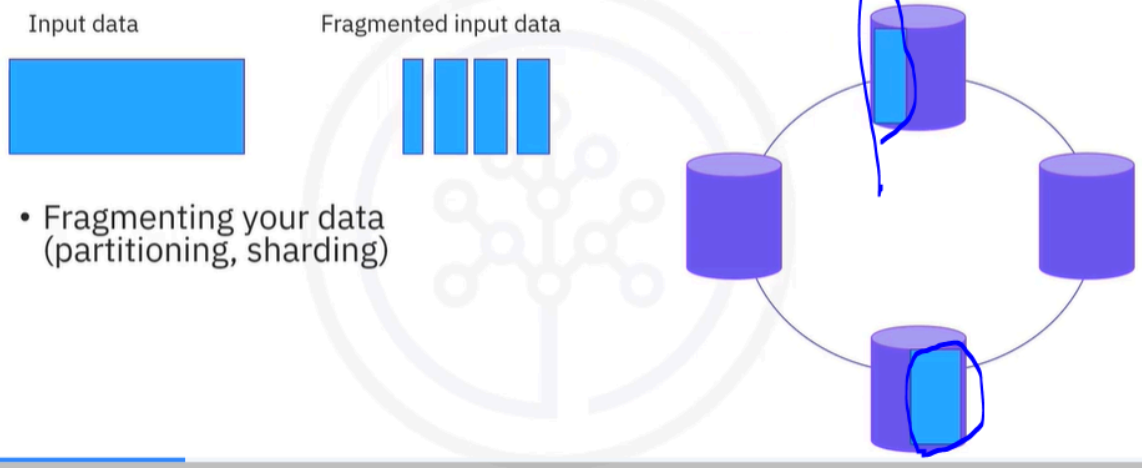
- A distributed database is a collection of multiple interconnected databases, which are spread physically across various locations that communicate via a computer network. A distributed database is physically distributed across the data sites by fragmenting and replicating the data. And a distributed database follows the base consistency model.

Fragmentation

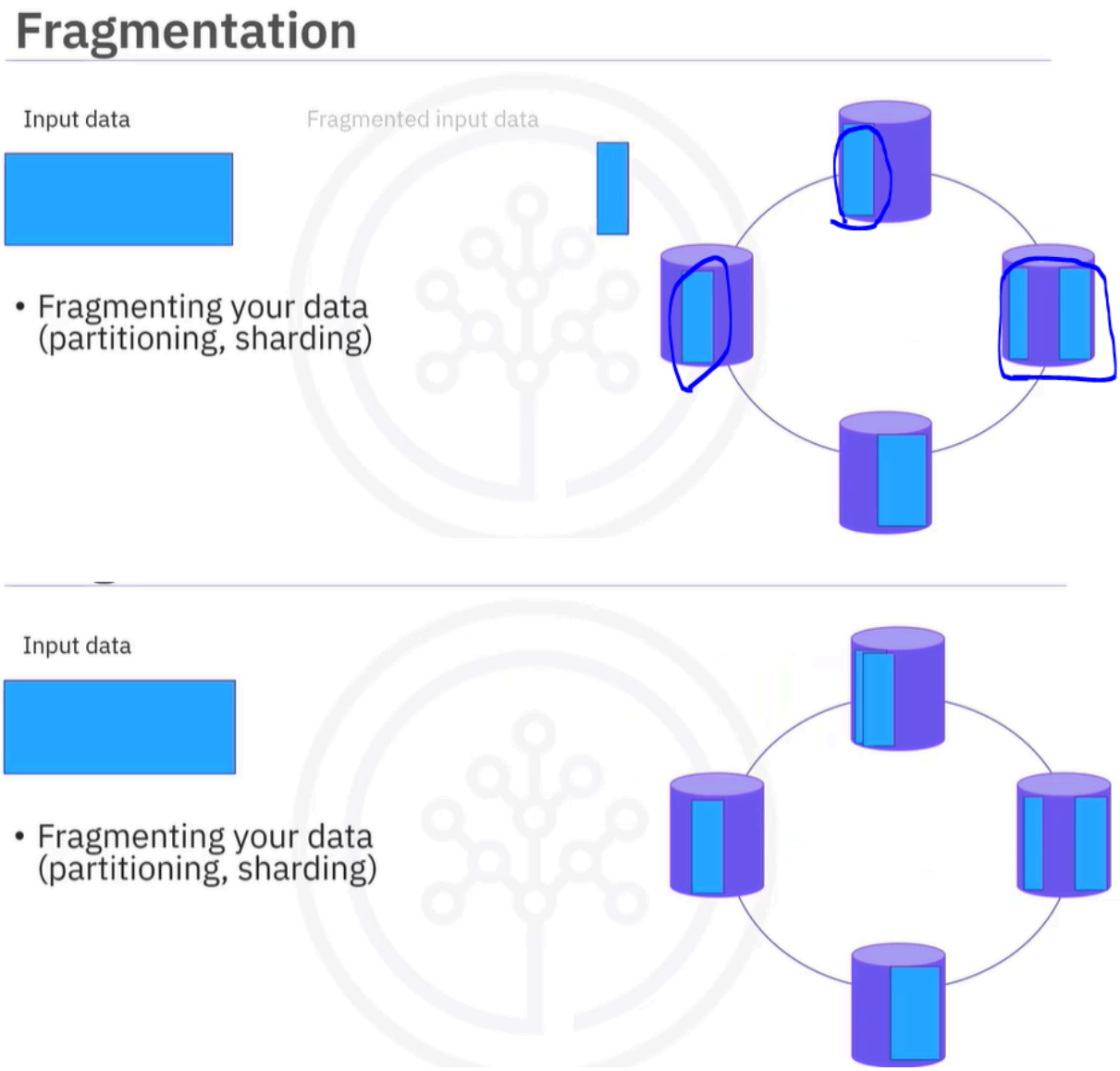


- Fragmenting your data (partitioning, sharding)

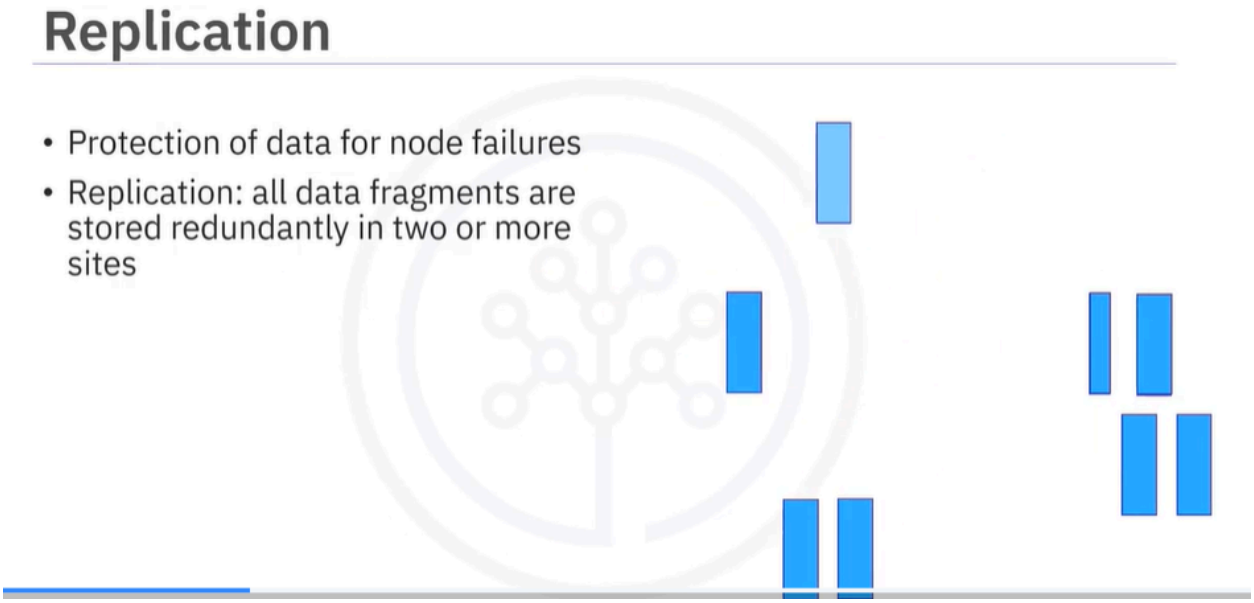
Fragmentation

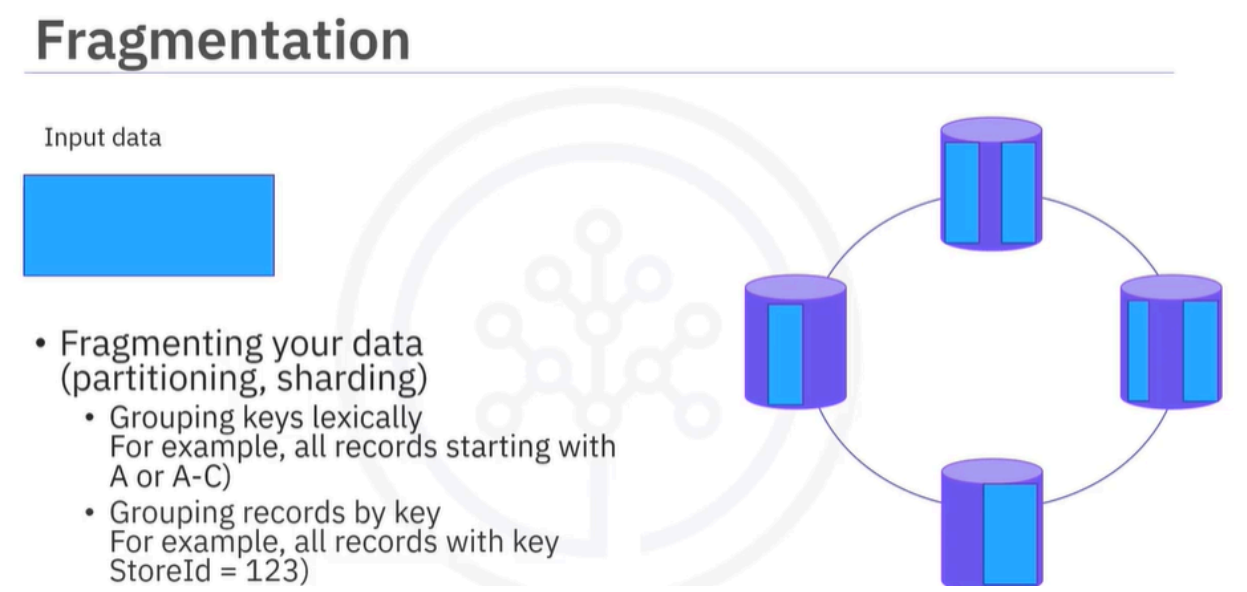


- Fragmenting your data (partitioning, sharding)

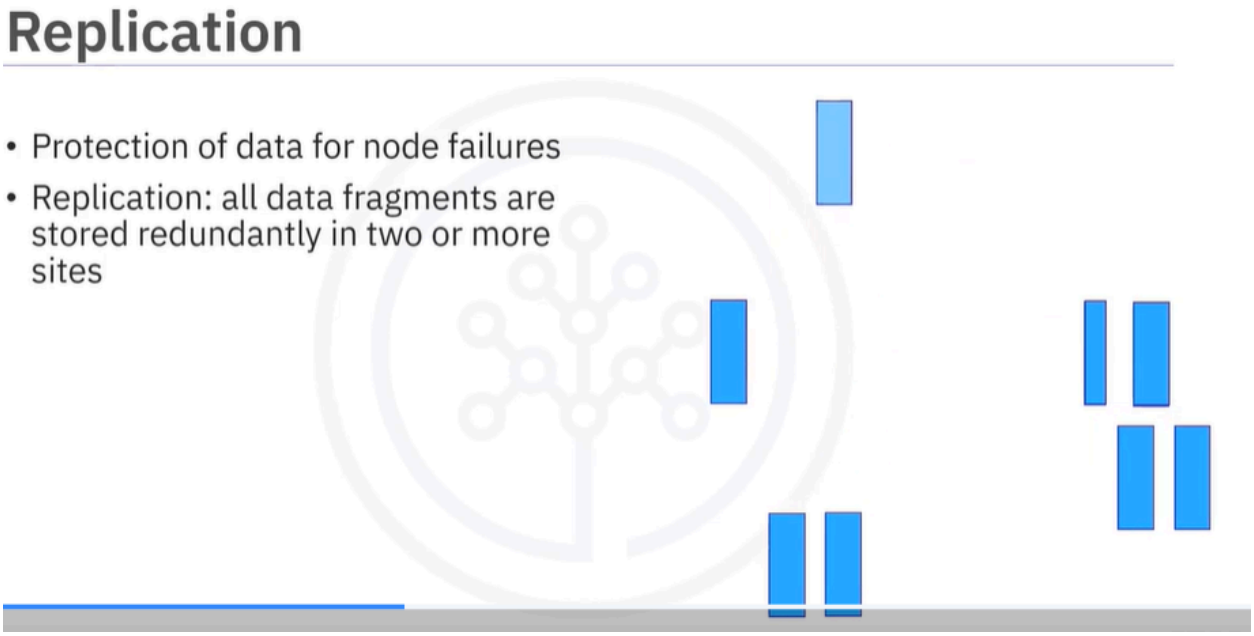


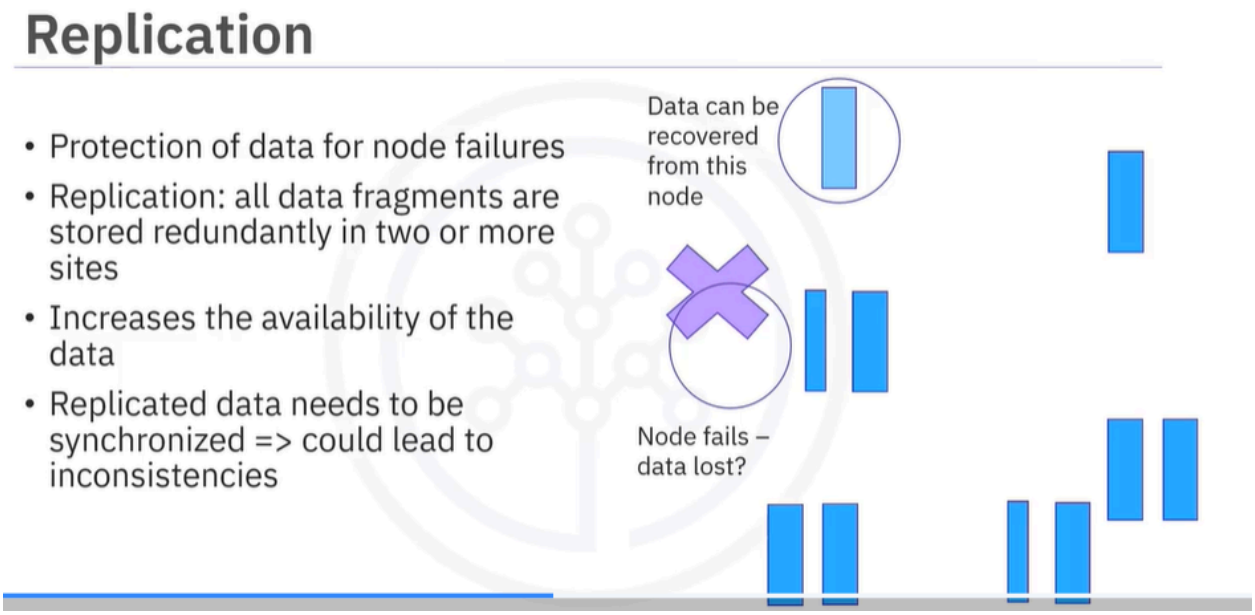
- To store a large piece of data on all servers of a distributed system, you need to break your data into smaller pieces. This process, also known as fragmentation of data, is also called partitioning of data or sharding of data by some known SQL databases, no matter the name, all distributed systems need to expose a way to store a large chunk of data.





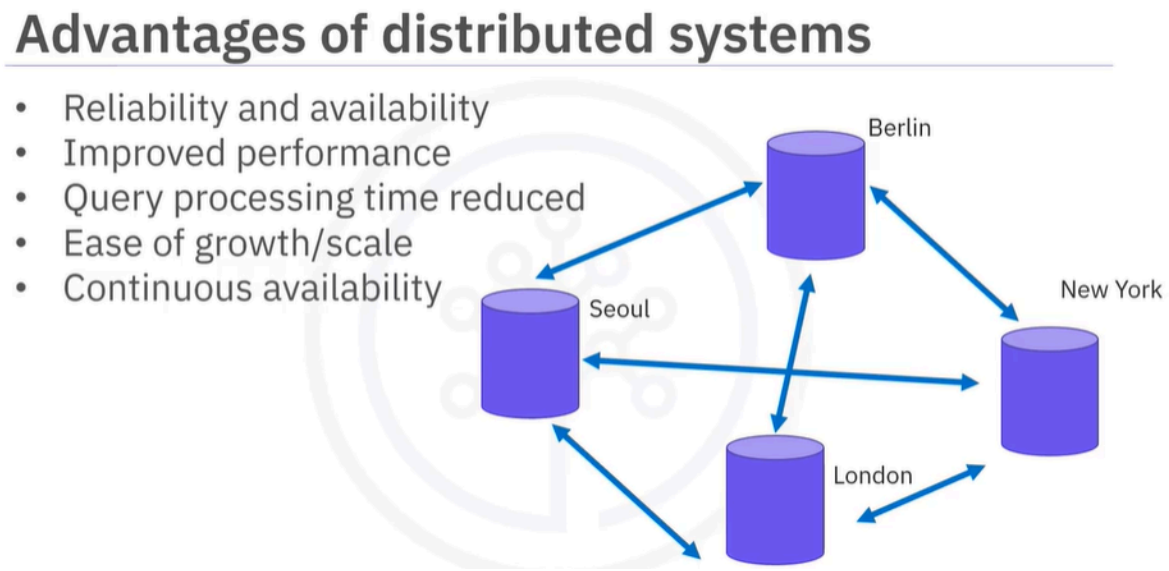
- This process is usually done by the key of the key value record in two ways. By either grouping all keys lexically. For example, all keys that start with A or between A and C can be found on a specific server, or by grouping all records that have the same key and placing them on the same server. For example, all transactions from a store where store ID is the key of the records. In this way with a query like give me all sales from a store, all records will be on a single server.



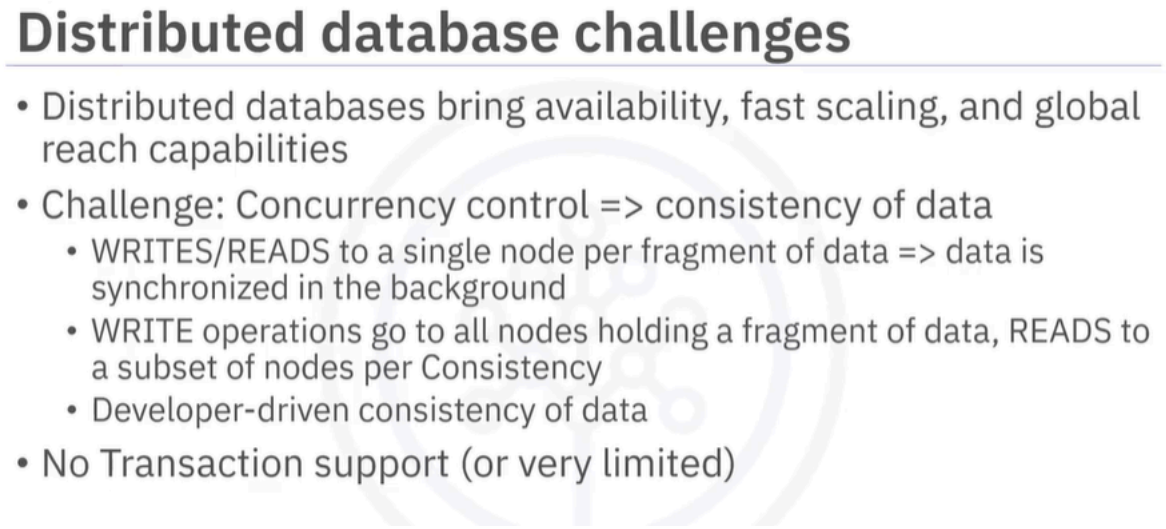


- Data is now distributed to all the clusters nodes. But how do we make sure that if a node fails we don't lose all of the data in that node? This is done through replication, where all fragments or partitions or shards of your data are stored redundantly in two or more sites. Hence, in replication, systems maintain copies of data. Replication increases the availability of data in different sites. If

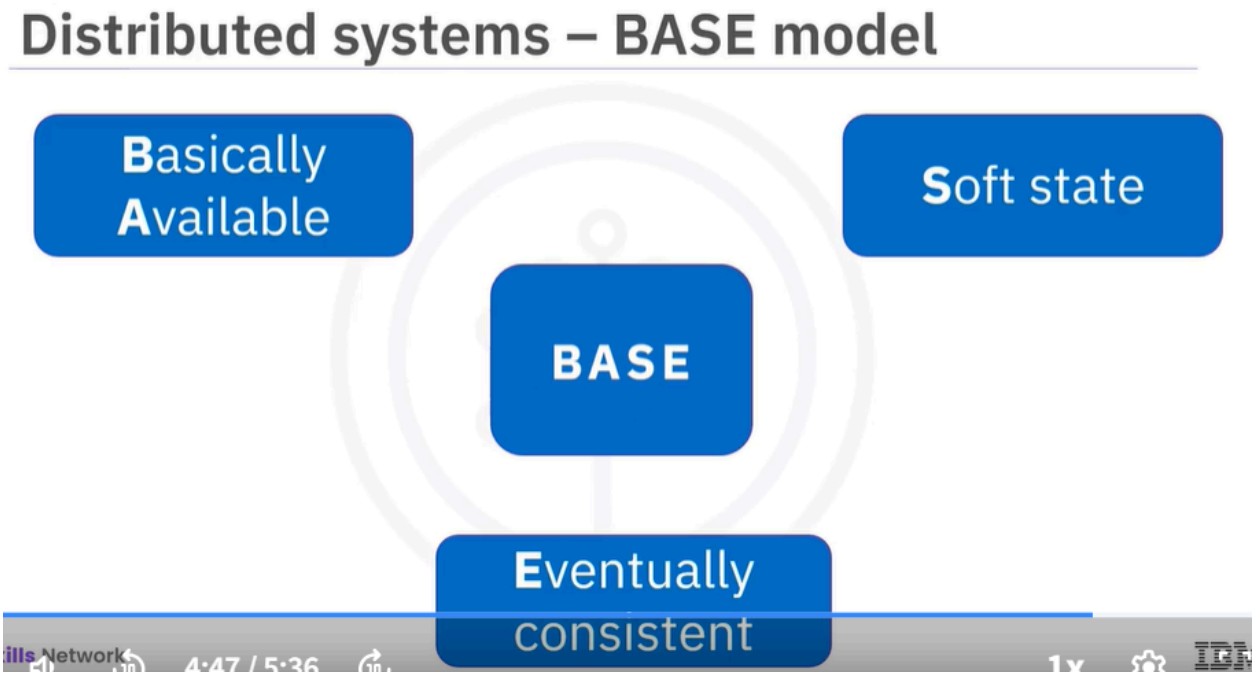
one node fails, that piece of data can be retrieved from another node. However, it has certain disadvantages as well. Data needs to be consistently synchronized. Any change made at one site needs to be replicated to every site where that related data is stored, or else it will lead to inconsistency.



- Distributed systems have numerous advantages. They allow more reliability and availability. The data is replicated at multiple sites. If the local server is unavailable, the data can be retrieved from another available server. Another advantage of distributed databases is improved performance. Especially for high volumes of data, query processing time is reduced, which also helps improve performance. You can easily grow or scale to increase your system capacity just by adding new servers to the cluster. Distributed systems also provide continuous operation with no more reliance on the central site.



- Distributed databases solve a lot of the technological issues for today's application services like availability, fast scaling, and global reach. But their architecture also introduces some challenges.
- One is concurrency control. Because the same piece of data is stored in multiple locations if you modify, update, or delete your data, how can data synchronization be secured? To solve this issue, some distributed databases direct operations for a certain fragment of data to only one node, leaving the cluster to synchronize with the other nodes.
- Others write to all nodes holding that particular fragment of data and read from as many nodes as required per consistency. In both cases, the developer can control the consistency of the operation or how many nodes need to answer for a certain operation to be considered successful. Due to concurrency control issues distributed databases by design don't really support transactions or provide a very limited version of them.



- Distributed databases follow the base model. Always available systems, data might be inconsistent at times, but eventually, consistency can be achieved.

What is a distributed database?

A **distributed database** is a type of database that is spread across multiple locations or servers, rather than being stored in a single central location. Here are the key points:

- **Multiple Locations**: The data is stored in different physical locations, which can be on different servers or even in different geographical areas.
- **Interconnected**: These locations are connected through a network, allowing them to communicate and share data with each other.
- **Fragmentation and Replication**: The data can be divided into smaller pieces (fragmentation) and stored in different locations. Additionally, copies of the data (replication) are kept in multiple places to ensure availability and reliability.

- **Advantages**: Distributed databases offer benefits like improved performance, reliability, and the ability to scale easily by adding more servers.
- **Challenges**: They also come with challenges, such as ensuring data consistency and managing complex operations across different locations.

In summary, a distributed database allows for efficient data management by spreading data across multiple sites while ensuring that it remains accessible and reliable.

How does fragmentation work in distributed databases?

Fragmentation in distributed databases is the process of breaking down a large dataset into smaller, manageable pieces called **fragments**. Here's how it works:

1. **Dividing Data**: The data is divided based on certain criteria. This can be done in a couple of ways:
    - **Horizontal Fragmentation**: This involves splitting the data into rows. For example, if you have a customer database, you might store all customers from a specific region in one fragment.
    - **Vertical Fragmentation**: This involves splitting the data into columns. For instance, you might keep customer names and addresses in one fragment and their purchase history in another.
2. **Storing Fragments**: Each fragment is then stored on different servers or locations. This allows for efficient data retrieval since queries can be directed to the specific server that holds the relevant fragment.
3. **Accessing Data**: When a query is made, the system knows where to find the specific fragments needed to fulfill that request. This reduces the amount of data that needs to be processed at once, improving performance.
4. **Benefits**: Fragmentation helps in:
    - **Improving Performance**: By reducing the amount of data each server has to handle.

- **Scalability**: Making it easier to add more servers as data grows.
- **Efficient Data Management**: Allowing for better organization of data based on access patterns.

In summary, fragmentation is a way to optimize data storage and access in distributed databases by breaking data into smaller, more manageable pieces.

What is the purpose of fragmentation in distributed databases?

The purpose of **fragmentation** in distributed databases includes several key benefits:

1. **Improved Performance**:
   - By breaking data into smaller pieces, queries can be processed more quickly since each server only needs to handle a portion of the data.
   - This reduces the load on any single server and speeds up data retrieval.
2. **Efficient Data Management**:
   - Fragmentation allows for better organization of data, making it easier to manage and access based on usage patterns.
   - It helps in distributing data according to how it is accessed, which can optimize performance.
3. **Scalability**:
   - As data grows, fragmentation makes it easier to add more servers to the system. New fragments can be created and distributed across additional servers without disrupting existing data.
   - This allows the system to scale horizontally, accommodating increasing data volumes.
4. **Reduced Network Traffic**:

- Since data is stored closer to where it is needed, fragmentation can minimize the amount of data that needs to be transferred over the network, reducing latency and improving response times.

5. **Enhanced Availability**:
    - By distributing fragments across multiple locations, the system can continue to operate even if one server fails. Other servers can still provide access to the remaining fragments.

In summary, fragmentation in distributed databases is essential for optimizing performance, managing data efficiently, enabling scalability, reducing network traffic, and enhancing availability.

what is the main purpose of replication?

The main purpose of **replication** in distributed databases is to ensure **data availability** and **reliability**. Here are the key points:

1. **Data Redundancy**:
    - Replication creates multiple copies of data across different servers or locations. This redundancy ensures that if one copy becomes unavailable, others can still provide access to the data.
2. **Increased Availability**:
    - By having copies of data in various locations, users can access the information even if one server fails. This enhances the overall availability of the system.
3. **Fault Tolerance**:
    - Replication helps the system continue functioning during hardware failures or network issues. If one server goes down, other replicas can still serve the data, minimizing downtime.
4. **Load Balancing**:

- With multiple copies of data, requests can be distributed across different servers. This reduces the load on any single server and improves response times for users.

5. **Data Consistency**:
    - While replication increases availability, it also requires mechanisms to keep the copies synchronized. This ensures that all replicas reflect the most current data, maintaining consistency across the system.

In summary, replication is essential for enhancing data availability, providing fault tolerance, and ensuring that users can access the data reliably.

How can replication improve data availability in a system?

Replication improves data availability in a system through the following mechanisms:

1. **Multiple Copies**:
    - By creating multiple copies of data across different servers or locations, replication ensures that if one copy becomes unavailable (due to server failure, maintenance, or network issues), other copies can still be accessed.
2. **Redundant Access Points**:
    - Users can access data from various servers. This redundancy means that even if one server is down, users can retrieve the data from another server, ensuring continuous access.
3. **Load Distribution**:
    - Replication allows for distributing user requests across multiple servers. This load balancing reduces the risk of any single server becoming a bottleneck, which can lead to downtime or slow response times.
4. **Geographical Distribution**:

- Data can be replicated across different geographical locations. This not only improves access speed for users in various regions but also provides resilience against localized failures (e.g., natural disasters).

5. **Automatic Failover**:
    - Many systems implement automatic failover mechanisms. If a primary server fails, the system can automatically redirect requests to a replica, ensuring that users experience minimal disruption.

In summary, replication enhances data availability by providing multiple access points, distributing load, and ensuring that data remains accessible even in the event of failures.

What challenges might arise with data replication in distributed systems?

Data replication in distributed systems can present several challenges, including:

1. **Data Consistency**:
    - Keeping all replicas synchronized can be difficult. If changes are made to one copy of the data, those changes must be propagated to all other copies. This can lead to inconsistencies if not managed properly.
2. **Latency**:
    - Replicating data across multiple locations can introduce latency. The time it takes for changes to be reflected in all replicas can affect the system's performance, especially in real-time applications.
3. **Network Overhead**:
    - Continuous replication requires bandwidth, which can lead to increased network traffic. This overhead can impact the performance of the system, especially if large volumes of data are being replicated frequently.
4. **Conflict Resolution**:

- In scenarios where multiple replicas can be updated independently, conflicts may arise. Determining which version of the data should be considered the "correct" one can be complex and may require additional mechanisms for conflict resolution.

5. **Increased Complexity**:
    - Implementing and managing replication adds complexity to the system architecture. Developers must consider how to handle failures, ensure consistency, and manage the overhead associated with replication.
6. **Scalability Issues**:
    - As the number of replicas increases, the complexity of maintaining them also grows. This can lead to challenges in scaling the system effectively while ensuring that all replicas remain consistent and available.

In summary, while data replication enhances availability and reliability, it also introduces challenges related to consistency, latency, network overhead, conflict resolution, complexity, and scalability.

What strategies could resolve conflicts in replicated data?

To resolve conflicts in replicated data, several strategies can be employed:

1. **Last Write Wins (LWW)**:
    - This strategy resolves conflicts by accepting the most recent update based on a timestamp. The version of the data with the latest timestamp is considered the valid one.
2. **Versioning**:
    - Each piece of data is assigned a version number. When conflicts occur, the system can use these version numbers to determine which version to keep or how to merge changes.
3. **Conflict-Free Replicated Data Types (CRDTs)**:

- CRDTs are data structures designed to automatically resolve conflicts in a way that guarantees eventual consistency. They allow concurrent updates without requiring coordination between replicas.

4. **Manual Resolution**:
    - In some cases, conflicts may require human intervention. Users can be notified of conflicts and given the option to choose which version to keep or how to merge changes.
5. **Application-Level Conflict Resolution**:
    - Applications can implement custom logic to handle conflicts based on the specific context of the data. This may involve business rules or domain-specific knowledge to determine the correct resolution.
6. **Quorum-Based Approaches**:
    - In quorum-based systems, a majority of replicas must agree on the value of the data before it is considered valid. This can help ensure that conflicting updates are resolved by requiring consensus among replicas.
7. **Synchronized Updates**:
    - In some systems, updates can be synchronized to ensure that only one replica can be updated at a time. This prevents conflicts from occurring in the first place but may reduce availability.

In summary, resolving conflicts in replicated data can be approached through various strategies, including last write wins, versioning, CRDTs, manual resolution, application-level logic, quorum-based approaches, and synchronized updates. Each strategy has its advantages and trade-offs, and the choice depends on the specific requirements of the system.